LAPORAN LAB STRUKTUR DATA

Rekursi dan Analisis Asimtotik

Nama: ..... NIM: .....

Catatan: kode lengkap terdapat pada berkas lab_rekursi.py. Bagian D menggunakan angka dari tangkapan layar yang diberikan; waktu dapat berbeda ketika eksperimen dijalankan ulang.

Bagian A — Rekursi Dasar

Fungsi	Kompleksitas waktu	Alasan singkat
A1 faktorial(n)	O(n)	Satu pemanggilan untuk setiap nilai n hingga base case.
A2 jumlah_digit(n)	O(log n)	Setiap langkah membagi n dengan 10; jumlah langkah sebanding jumlah digit.
A3 balik_string(s)	O(n²)	Ada n pemanggilan dan slicing/konkatenasi string berbiaya linear.
A4 is_palindrom(s)	O(n²)	Slicing s[1:-1] membuat string baru pada setiap tingkat rekursi.
A5 pangkat_cepat(x,n)	O(log n) perkalian	Eksponen dibagi dua di setiap pemanggilan; ruang stack O(log n).

A tertulis 1 — Call stack faktorial(4): faktorial(4) menunggu 4 × faktorial(3); faktorial(3) menunggu 3 × faktorial(2); faktorial(2) menunggu 2 × faktorial(1); faktorial(1) mencapai base case dan mengembalikan 1. Nilai kembali: faktorial(2)=2, faktorial(3)=6, faktorial(4)=24.

A tertulis 2 — Base case dihapus: rekursi tidak memiliki kondisi berhenti dan terus memanggil faktorial(n−1) sampai batas rekursi Python tercapai. Galat yang muncul adalah RecursionError: maximum recursion depth exceeded (pesan dapat sedikit berbeda antarversi Python).

A tertulis 3 — Mengapa O(n²): slicing s[1:] menyalin string baru dengan panjang yang berkurang satu setiap langkah. Total karakter yang disalin kira-kira (n−1)+(n−2)+...+1 = n(n−1)/2, sehingga waktu total O(n²), bukan O(n).

Bagian B — Rekursi Lanjut

Fungsi	Kompleksitas waktu	Alasan singkat
B1 fib_naif(n)	O(2■)	Banyak submasalah Fibonacci dihitung berulang.
B2 fib_memo(n)	O(n)	Setiap nilai Fibonacci disimpan dan dihitung paling banyak sekali.
B3 binary_search_rek	O(log n)	Ruang pencarian dibelah dua setiap pemanggilan.
B4 permutasi(s)	O(n · n!)	Menghasilkan n! string, masing-masing panjang n.
B5 hanoi(n,...)	O(2■)	Relasi T(n)=2T(n−1)+O(1); jumlah langkah 2■−1.
B6 ratakan(data)	O(N) secara ideal	N adalah jumlah elemen total; implementasi list slicing/concatenation dapat menambah biaya salin.

B tertulis 1 — Pohon rekursi fib_naif(5):

```
fib(5)
■■■ fib(4)
■   ■■■ fib(3)
■   ■   ■■■ fib(2)
■   ■   ■■■ fib(1)
■   ■■■ fib(2)
■■■ fib(3)
    ■■■ fib(2)
    ■■■ fib(1)
```

fib_naif(2) dipanggil 3 kali pada pohon tersebut.

B tertulis 2 — Memoisasi: tanpa memo, cabang rekursif menghitung nilai yang sama berulang kali sehingga kompleksitas eksponensial O(2■). Dengan dictionary memo, setiap nilai F(0) sampai F(n) dihitung satu kali; total waktu dan ruang menjadi O(n).

B tertulis 3 — Hanoi(20): jumlah langkah = 2²■ − 1 = 1.048.575 langkah. Jika satu langkah membutuhkan 1 detik, totalnya 1.048.575 detik = 12 hari, 3 jam, 12 menit, 15 detik.

Bagian C — Analisis Asimtotik

Soal	Relasi/perhitungan	Jawaban Big-O
C1	n iterasi	O(n)
C2	n × n iterasi	O(n²)
C3	0+1+...+(n–1) = n(n–1)/2	O(n²)
C4	i dikali 2 tiap iterasi; sekitar log■ n iterasi	O(log n)
C5	n iterasi luar × log■ n iterasi dalam	O(n log n)
C6	Dua loop terpisah, n+n	O(n)
C7	T(n)=T(n–1)+O(1)	O(n)
C8	T(n)=T(■n/2■)+O(1)	O(log n)
C9	T(n)=2T(n–1)+O(1)	O(2■)
C10	T(n)=2T(n/2)+O(n), sesuai asumsi penggabungan pada soal	O(n log n)
C11	T(n)=2T(n/2)+O(1)	O(n)
C12	Urutan: 1 < log n < √n < n < n log n < n² < 2■ < n!	—
C13	Pilih c=15 dan n■=1	5n²+3n+7 ≤ 15n² untuk n≥1

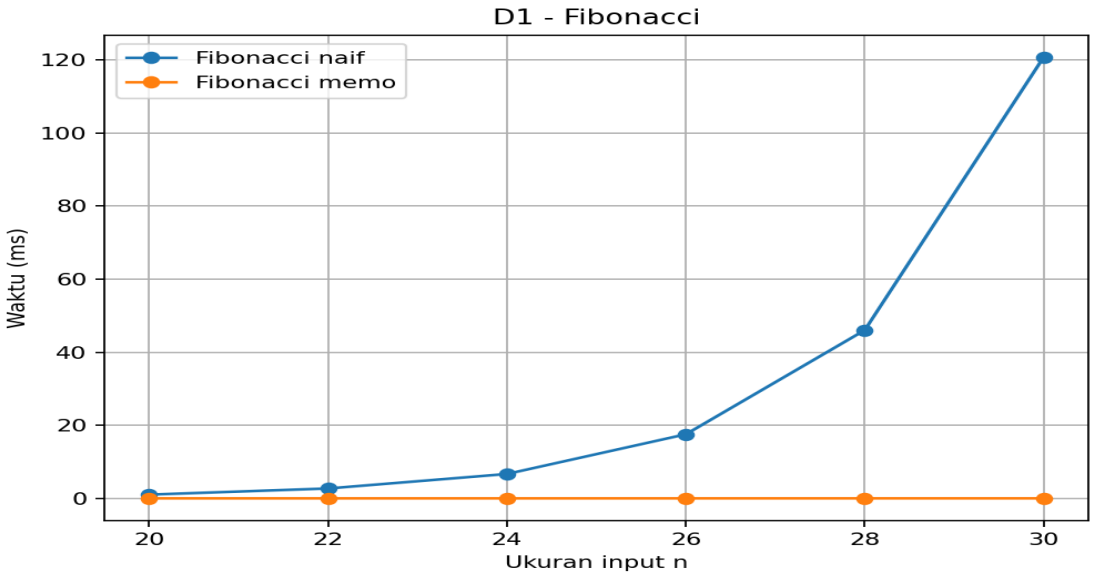
C13 pembuktian: Untuk n ≥ 1, berlaku 3n ≤ 3n² dan 7 ≤ 7n². Maka 5n² + 3n + 7 ≤ 5n² + 3n² + 7n² = 15n². Jadi, berdasarkan definisi Big-O, c=15 dan n■=1 memenuhi syarat.

Bagian D — Eksperimen Empiris

Tabel berikut menyalin hasil yang tampak pada tangkapan layar pengguna. Satuan waktu adalah milidetik (ms). Pengukuran baru dapat berbeda karena perangkat dan kondisi proses.

n	fib_naif (ms)	fib_memo (ms)
20	1.0298	0.0041
22	2.7108	0.0039
24	6.6750	0.0045
26	17.5368	0.0060
28	46.0260	0.0064
30	120.7271	0.0070

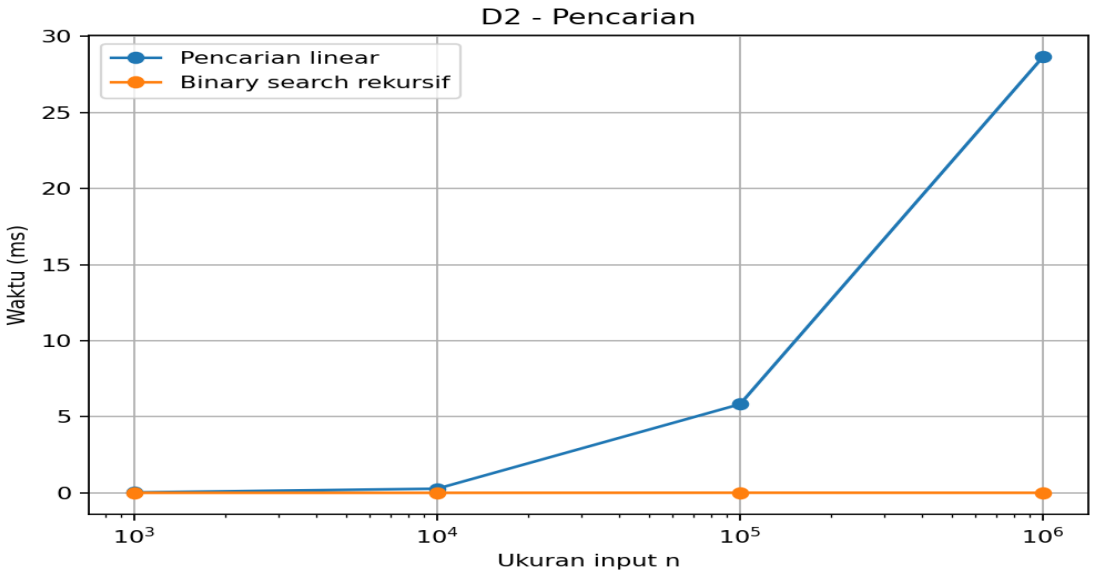
Grafik D1 — Fibonacci



D1 analisis: rasio waktu fib_naif ketika n naik 2 berturut-turut sekitar 2,63; 2,46; 2,63; 2,62; dan 2,62. Nilai ini mendekati $\phi^2 \approx 2{,}618$, walaupun tidak harus sama persis karena variasi pengukuran. fib_memo tetap sangat cepat dan tumbuh jauh lebih lambat.

n	linear (ms)	binary_rek (ms)
1000	0.0297	0.0019
10000	0.2724	0.0018
100000	5.8257	0.0083
1000000	28.6458	0.0039

Grafik D2 — Pencarian

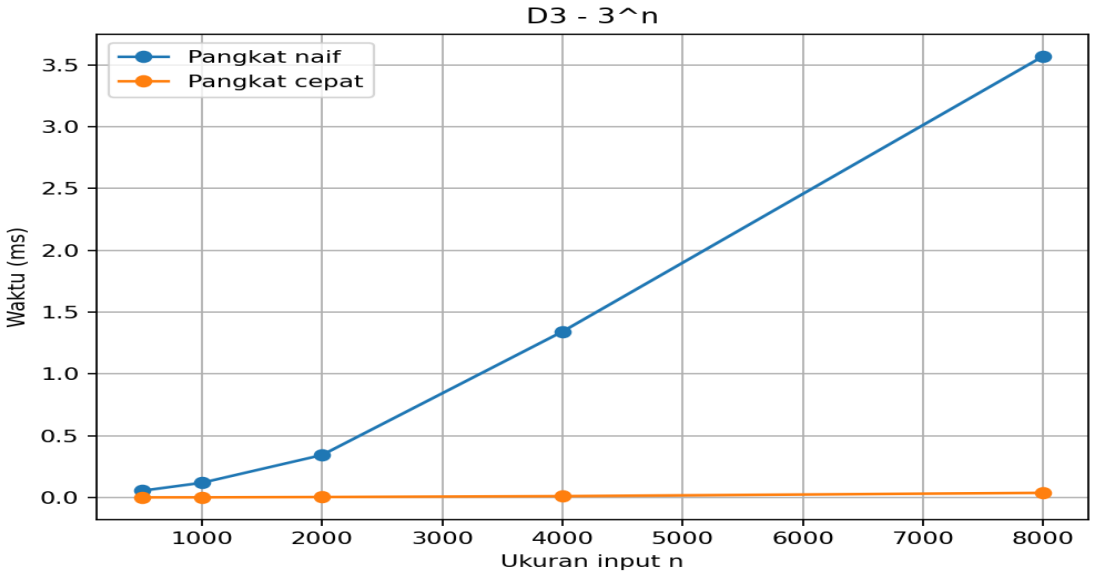


D2 analisis: saat n bertambah 10 kali, pencarian linear secara teori dapat membutuhkan waktu sekitar 10 kali lebih lama (O(n)), sedangkan binary search hanya bertambah sekitar logaritmik (O(log n)). Angka pada screenshot tidak meningkat secara konsisten—khususnya binary search—karena waktu yang sangat kecil rentan noise, resolusi timer, cache, dan beban sistem. Ulangi eksperimen jika perlu pembandingan yang lebih stabil.

n	pangkat_naif (ms)	pangkat_cepat (ms)
500	0.0553	0.0020
1000	0.1198	0.0022

n	pangkat_naif (ms)	pangkat_cepat (ms)
2000	0.3445	0.0047
4000	1.3431	0.0118
8000	3.5709	0.0385

Grafik D3 — Pangkat 3^n



D3 analisis: pangkat cepat memakai O(log n) perkalian, tetapi perkalian bilangan bulat Python bukan O(1) untuk bilangan yang terus membesar. 3■■■■ memiliki sekitar 3.817 digit desimal (floor(8000 log■■3)+1). Operasi aritmetika pada bilangan sebesar ini membutuhkan waktu lebih besar daripada operasi pada bilangan kecil.

D4 — Mengapa hasil pengukuran berbeda: spesifikasi CPU, RAM, versi Python, proses latar belakang, cache, pemanasan interpreter, presisi timer, dan variasi sistem operasi dapat memengaruhi waktu. Big-O menggambarkan tren pertumbuhan saat n besar, bukan waktu absolut pada satu percobaan.

Catatan Pengumpulan

Sebelum dikumpulkan, jalankan python uji_lab.py di folder yang sama dengan lab_rekursi.py. File uji_lab.py tidak disertakan dalam lampiran yang diterima di percakapan ini, sehingga status 11/11 perlu dikonfirmasi dengan penguji milik dosen. Kode ini mengikuti spesifikasi fungsi pada PDF, tetapi tetap periksa kompatibilitas dengan kerangka/penguji yang disediakan dosen.